



UNIVERSIDAD
DON BOSCO

Programación Orientada a Objetos en Java: Fundamentos Esenciales

Un recorrido completo por los conceptos fundamentales de Java que transformarán tu forma de programar.



Introduccion

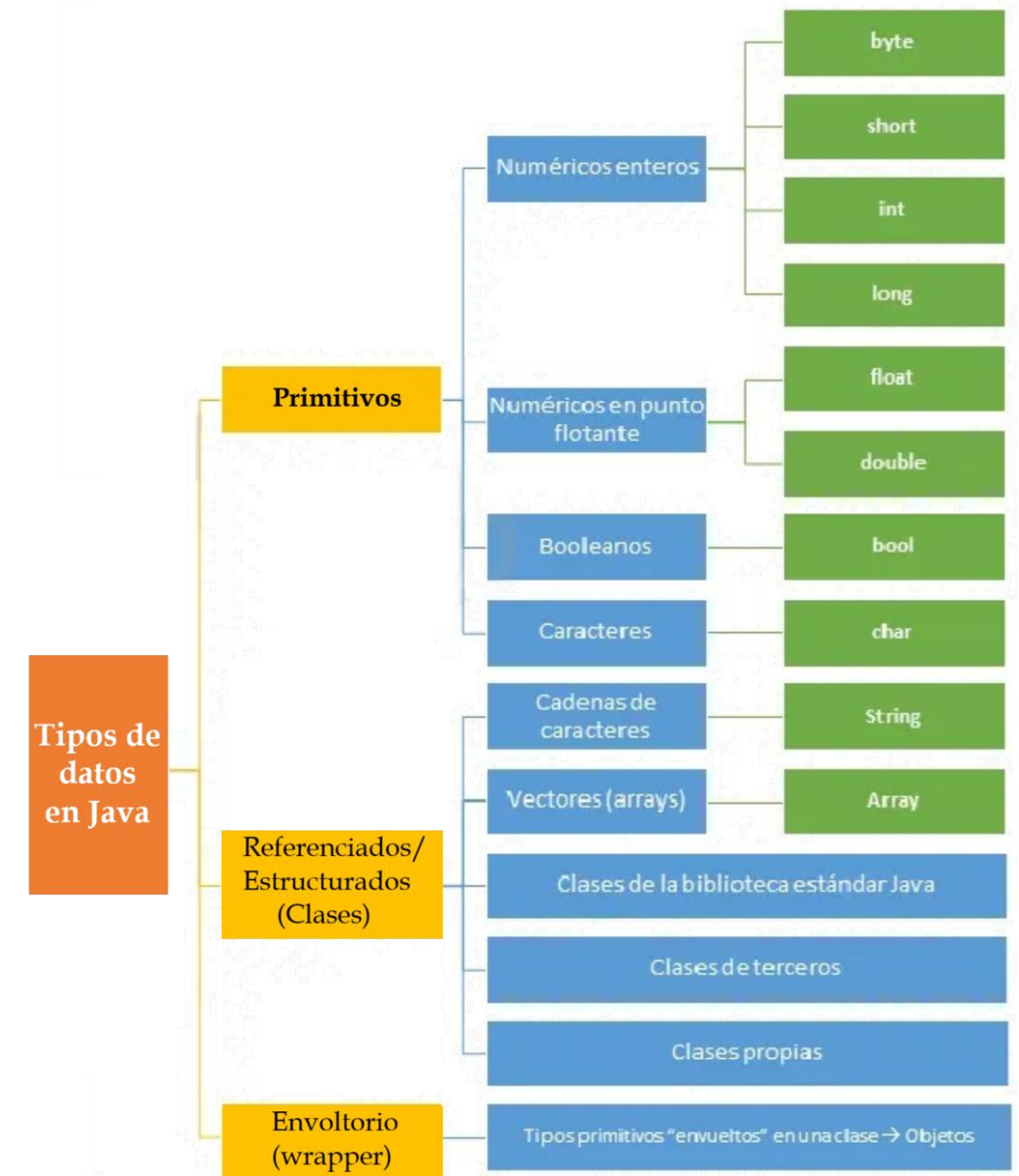
Para introducirnos al tema de Clases, es necesario recordar la clasificacion de los tipos de datos utilizada por Java, la cual se muestra en la figura a la derecha.

Hasta ahora se han utilizado los tipos de datos basicos (***Primitivos***).

En Java el concepto de Clases es fundamental para poder manejar este lenguaje de programacion de manera adecuada.

Una clase define la naturaleza de un objeto, por lo tanto, una clase provee las bases de la programación orientada a objetos. Así que cualquier concepto que se vaya a implementar en Java está envuelto en una clase.

En este tema veremos como crear una clase y a su vez cómo podemos crear objetos a partir de nuestra clase y utilizarlos.



Clase y Objeto: Los Pilares de la POO (1)

¿Qué es una Clase?

Una **clase** es un **plano o plantilla** que define la estructura y el comportamiento de un tipo de objeto y con la cual se crearán múltiples instancias. La clase especifica qué datos tendrá el objeto (llamados **atributos** o **campos**) y qué operaciones podrá realizar (llamados **métodos**).

Las clases permiten modelar entidades del mundo real, agrupando datos relacionados y la lógica que opera sobre esos datos en una sola unidad cohesiva. Se declaran usando la palabra clave **class** y siguen la convención de nombres **PascalCase**.



La Clase es el Plano

Define la estructura, propiedades y comportamientos generales

¿Qué es un Objeto?

Un **objeto** es una instancia concreta y real de una clase. Mientras que la clase define la estructura general (la receta), cada objeto es una manifestación única de esa clase, con sus propios valores específicos para los atributos definidos y su propio estado.

Por ejemplo, si **Automovil** es una clase, entonces **miToyotaCorolla** o **tuHondaCivic** son objetos específicos de esa clase. Cada objeto se crea usando el operador **new** y existe independiente en la memoria, manteniendo su propio *estado y comportamiento*.



El Objeto es la Construcción.

Instancia concreta con valores específicos y estado propio



Múltiples Instancias.

Cada objeto es único aunque comparte la misma estructura y métodos

Clase y Objeto: Los Pilares de la POO (2)



Clase, un tipo de dato en Java

- Una clase genera un nuevo tipo de datos en Java y lo utilizamos para crear objetos de este tipo definido.
- Por lo tanto, una clase es una plantilla para poder crear objetos.
- Debido a que un objeto es una instancia de una clase, normalmente utilizaremos “**objeto**” o “**instancia de una clase**” de manera equivalente.

¿Donde se definen las nuevas Clases?

Una clase se define en un archivo independiente de código (**.java**), el cual debe almacenarse con el mismo nombre de la Clase que define.

La clase se inicia con un encabezado, utilizando la palabra **class** y luego el nombre de la clase que definirá. Luego, con una pareja de llaves se define al cuerpo de esta clase, que contendrá la definición de sus miembros (campos, métodos y otros).

Es importante aclarar que en un archivo de código (**.java**) se pueden definir múltiples clases, pero solo una de ellas puede ser *public*, y esa clase pública debe tener el mismo nombre que el archivo (ej. `MiClasePrincipal.java`).

Las otras clases definidas en el mismo archivo `.java` no serán públicas (pueden ser marcadas como `default` o `package-private`) y funcionan como clases de apoyo.

Clase y Objeto: Los Pilares de la POO (3)

Ejemplo de definicion de clase MiClase

- Su archivo debera llamarse **MiClase.java**
- Las llaves definen el cuerpo de la clase, que incluire a miembros (propiedades y comportamientos generales).
- Cada método debe llevar su correspondiente pareja de llaves, para definir a su cuerpo de instrucciones a ejecutar.

```
public class MiClase {  
    //atributos|campos de la clase  
    private String atributo1;  
    private int atributo2;  
    //...  
  
    //metodos de la clase  
    //constructor: metodo con mismo nombre de clase  
    public MiClase() {  
    }  
  
    //otros metodos de la clase  
    public void metodo1() {  
    }  
    //...  
}
```

Clase y Objeto: Los Pilares de la POO (4)



Utilizacion de paquetes para organizar Clases

No es obligatorio definir una clase dentro de un paquete.

Si no declara una clase dentro de un paquete, esta clase se integra a un paquete por defecto (o *default package*).

Sin embargo, utilizar paquetes es una de las mejores prácticas en programación en Java, por los siguientes motivos:

- **Organización:** Permite agrupar clases con funcionalidades afines (por ejemplo, todas las clases de interfaz gráfica en un paquete y las de conexión a base de datos en otro).
- **Control de acceso:** Facilita el encapsulamiento. Implementa niveles de visibilidad que restringen qué clases pueden interactuar con otras.
- **Evitar conflictos de nombres:** Permite tener dos clases con el mismo nombre, pero definidas en diferentes paquetes.

Clase y Objeto: Los Pilares de la POO (5)

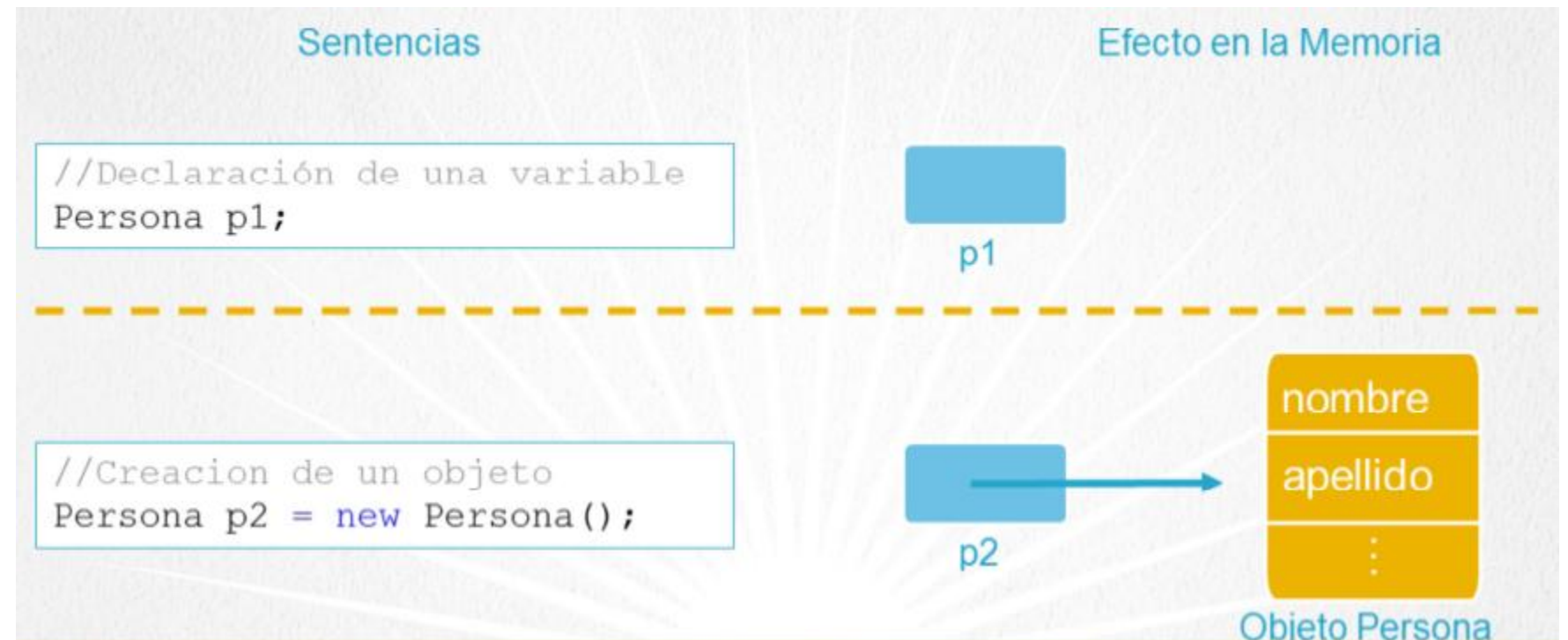
Creacion de Objetos (instancias de clases)

- Un objeto es la instancia de una clase, es decir que a partir de nuestra Clase (plantilla), se crean objetos con los cuales podremos trabajar, es decir, cambiar sus valores y llamar a sus métodos.
- La palabra reservada **new** se emplea para crear nuevos objetos, instancias de una determinada clase que indicamos a continuación seguida de un par de paréntesis.

- Ejemplo de declaración y creación de un objeto:

```
//Declaracion de un objeto Persona
Persona p1;
//Instanciar la clase Persona
p1=new Persona();
```

- La palabra **new** permite solicitar memoria para los datos que almacenara el objeto, como lo muestra la imagen:



Miembros que conforman a una Clase

Los miembros de una clase son los elementos que la componen y permiten crear objetos con características y acciones comunes, permitiendo la organización y reutilización del código mediante la encapsulación de datos y funcionalidades.

Tipos de miembros

- **Campos o Atributos:** Variables que almacenan el **Estado o las características** de un objeto (ej. marca, modelo, precio).
- **Métodos:** Funciones que definen las **acciones o comportamientos** que un objeto puede realizar (ej. acelerar(), frenar(), imprimirDatos()).
- **Constructores:** Métodos especiales con el mismo nombre que la clase, usados para inicializar los objetos al crearlos.
- **Miembros Estáticos:** Pertenecen a la clase misma y no a instancias individuales (objetos). Se comparten entre todos los objetos.



Características

- Marca
- Color
- Tipo
- Precio
- No. de Puertas
- Tipo Combustible
- Cilindros
- Transmisión

Acciones

- Encender
- Avanzar
- Retroceder
- Detener
- Apagar

Características

- Nombre
- Especie
- Color
- Edad

Acciones

- Comer
- Dormir
- Correr

Modificadores de Acceso en Java: Controlando la Visibilidad

Los modificadores de acceso son palabras clave que establecen el nivel de *visibilidad y accesibilidad* de las clases y sus miembros (campos/atributos, métodos y constructores).

Implementan el principio de encapsulamiento, uno de los pilares de la programación orientada a objetos en Java.

1 **public** - Acceso Universal

El miembro es accesible desde cualquier parte del programa Java, sin restricciones.

- Visible desde cualquier clase en cualquier paquete.
- Ideal para la API pública de un componente.
- Máxima exposición; debe usarse con responsabilidad.

3 **default** (sin modificador) - Acceso de Paquete

Cuando no se especifica ningún modificador de acceso, el miembro es accesible solo dentro de las clases del mismo paquete Java.

Es útil para organizar componentes que colaboran estrechamente dentro de un módulo.

- Visible solo para clases dentro del mismo **package**.
- No usa ninguna palabra clave explícita para declararse.
- Fomenta la cohesión de módulos internos y oculta detalles de implementación.

2 **protected** - Acceso para Herencia

Accesible dentro del mismo paquete y por todas las subclases, incluso si estas subclases están en diferentes paquetes.

Permite que las clases hijas extiendan o modifiquen el comportamiento base.

- Acceso dentro del mismo paquete (**package-private**).
- Disponible para clases hijas (subclases) a través de la herencia.
- Ofrece un balance entre protección y flexibilidad en jerarquías de herencia.

4 **private** - Acceso Exclusivo

El nivel más restrictivo en Java. El miembro es accesible solo dentro de la propia clase donde fue declarado. De esta manera, se gestiona la lógica interna de la clase.

- Máxima encapsulación; oculta por completo la implementación.
- Previene modificaciones externas directas de los datos internos.
- Esencial para el control estricto del estado de un objeto.

Modificadores de Acceso en Java: Controlando la Visibilidad

Modificadores de acceso

MODIFICADOR	CLASE	PACKAGE	SUBCLASE	USO NO HEREDADO
public	Sí	Sí	Sí	Sí
protected	Sí	Sí	Sí	No Sí en el mismo paquete
default	Sí	Sí	No Sí en el mismo paquete	No Sí en el mismo paquete
private	Sí	No	No	No

Ejemplo: Clase y Objeto



```
// Definición de una clase en Java: Automovil
public class Automovil {
    // Atributos (campos) de la clase, siguiendo camelCase
    public String marca;
    public String modelo;

    public Automovil(String marca, String modelo) { // Constructor de clase para inicializar objetos
        this.marca = marca;
        this.modelo = modelo;
    }

    public void mostrarDetalles() { // Método para mostrar los detalles del automóvil
        System.out.println("Marca: " + marca + ", Modelo: " + modelo);
    }

    // Método main para demostrar la creación y uso de objetos
    public static void main(String[] args) {
        // Creando objetos (instancias) de la clase Automovil y asignando valores específicos
        Automovil miToyotaCorolla = new Automovil("Toyota", "Corolla");
        Automovil tuHondaCivic = new Automovil("Honda", "Civic");
        Automovil elFordFocus = new Automovil("Ford", "Focus");
        // Llamando a métodos en los objetos
        System.out.print("Mi Auto: ");
        miToyotaCorolla.mostrarDetalles(); // Salida: Marca: Toyota, Modelo: Corolla
        System.out.print("Tu Auto: ");
        tuHondaCivic.mostrarDetalles(); // Salida: Marca: Honda, Modelo: Civic
        System.out.print("El Auto: ");
        elFordFocus.mostrarDetalles(); // Salida: Marca: Ford, Modelo: Focus
    }
}
```

Campos de una clase:

Almacenamiento de datos del objeto

Los **campos**, también conocidos como **atributos** o **variables de instancia**, son las características o valores que definen el *estado de un objeto* en Java. Almacena los datos que el objeto necesita para funcionar correctamente. El uso adecuado de campos es fundamental para implementar el *principio de encapsulamiento* en la Programación Orientada a Objetos.

Estado del Objeto

Los campos almacenan información específica de cada instancia.

Cada objeto mantiene sus propios valores, independientes de otros objetos de la misma clase.

- Definen las características del objeto
- Pueden ser de cualquier tipo de dato Java (primitivos o referencias)
- Persisten durante la vida útil del objeto.

Inicialización

Los campos en Java pueden tener valores por defecto, ser inicializados explícitamente en su declaración, o a través de constructores, asegurando que el objeto siempre esté en un estado válido.

- Valores predeterminados del lenguaje para tipos primitivos.
- Inicialización explícita en la declaración del campo.
- Asignación de valores mediante constructores de la clase

Tipos de Campos en Java

```
import java.util.ArrayList; // Ejemplo de importación
import java.util.Date;      // Ejemplo de importación
public class CuentaBancaria {
    // Campos de instancia (privados para encapsulamiento)
    private String numeroCuenta;
    private String titular;
    private double saldo;
    private String tipoCuenta;
    private boolean activa;
    // Campo inicializado explícitamente y con convención de nombre Java
    private int transaccionesRealizadas = 0;
    // Constante estática (compartida por todas las instancias)
    public static final double COMISION_POR_TRANSACCION = 0.02;
    // Constructor para inicializar campos
    public CuentaBancaria(String numeroCuenta, String titular,
                           double saldoInicial) {
        this.numeroCuenta = numeroCuenta;
        this.titular = titular;
        this.saldo = saldoInicial;
        this.tipoCuenta = "Ahorro"; // Valor por defecto
        this.activa = true;         // Valor por defecto
    }
    public void depositar(double monto) {
        if (monto > 0) {
            this.saldo += monto;
            this.transaccionesRealizadas++;
            System.out.println("Depósito de " + monto + " realizado.");
        }
    }
}
```

- **Primitivos:** `int`, `double`, `boolean`, `char`, `byte`, `short`, `long`, `float`.
- Almacenan valores directamente.
- **Objetos (Tipos de Referencia):** `String`, `Date`, `ArrayList`.
- Almacenan referencias a objetos en memoria.
- **Constantes:** Declarados con la palabra clave `final`.
- Su valor no puede cambiar después de la inicialización, ej.
`public static final double PI = 3.14159;`
- **Estáticos:** Declarados con `static`.
- Pertenecen a la clase, no a una instancia particular.
- Se comparten por todas las instancias de la clase.

Ejemplo Práctico en Java

```
import java.util.ArrayList; // Ejemplo de importación
import java.util.Date;     // Ejemplo de importación

public class CuentaBancaria {
    // Campos de instancia (privados para encapsulamiento)
    private String numeroCuenta;
    private String titular;
    private double saldo;
    private String tipoCuenta;
    private boolean activa;
    private int transaccionesRealizadas
    // Constante estática (compartida por todas las instancias)
    public static final double COMISION_POR_TRANSACCION = 0.02;

    // Constructor para inicializar campos
    public CuentaBancaria(String numeroCuenta, String titular, double saldoInicial)
    {
        this.numeroCuenta = numeroCuenta;
        this.titular = titular;
        this.saldo = saldoInicial;
        this.tipoCuenta = "Ahorro"; // Valor por defecto
        this.activa = true;         // Valor por defecto
        transaccionesRealizadas = 0;
    }

    // Ejemplo de método (no requerido para esta tarjeta, pero ilustrativo)
    public void depositar(double monto) {
        if (monto > 0) {
            this.saldo += monto;
            this.transaccionesRealizadas++;
            System.out.println("Depósito de " + monto + " realizado.");
        }
    }
}
```



Mejores Prácticas en Java

- **Campos (atributos):** Casi siempre `private`. Accede y modifica su valor a través de métodos `public` `getters` y `setters` (que pueden incluir validación).
- **Métodos auxiliares:** `private` si solo se usan internamente dentro de la clase para implementar su lógica.
- **API Pública de la Clase:** `public` solo para aquellos métodos y constantes que otras clases realmente necesitan usar o invocar.
- **Diseño para Herencia:** Utiliza `protected` para miembros que las subclases deberían poder acceder o modificar, pero que no son parte de la interfaz pública general.
- **Cohesión de Paquetes:** El acceso `default` es útil para clases, campos o métodos que necesitan interactuar estrechamente dentro de un mismo paquete, pero deben permanecer ocultos al exterior.
- **Constantes:** A menudo se declaran como `public static final` para ser accesibles globalmente y representar valores fijos.

El uso correcto de modificadores de acceso en Java protege tus datos, facilita el mantenimiento del código, previene errores y crea interfaces de clases claras, seguras y fáciles de usar para otros desarrolladores.

Métodos de una clase:

Comportamiento de los objetos de una misma clase

- Los métodos de una clase son las **funciones/procedimientos** que definen las acciones (**Comportamientos**) que sus objetos pueden realizar, interactuando con sus atributos (datos) para obtener información o ejecutar tareas, como calcularIMC o prestarLibro.
- A las acciones que ejecutan los metodos tambien se les conoce como *Logica de Negocios*.
- Son fundamentales para la modularidad y reutilización del código.

Llamada a un Metodo

- Una vez agregado un método a una clase, podemos hacer uso de él haciendo lo siguiente:
- 1) Declarar un objeto del tipo de la clase que tiene el método a utilizar.
- 2) Por medio del objeto declarado, se utiliza el operador (.) y luego el nombre del método.

Llamada general de un método en Java

```
//Crear un objeto de la clase a llamar su método
TipoClase objeto = new TipoClase();

//Llamamos el método, enviando argumentos si se requieren
//Si el método regresa un valor podemos recibirlo según el tipo
tipoDevuelto resultado = objeto.nombreMetodo(arg1, arg2, etc);
```

Ejemplo de llamada a un método:

```
//Creamos un objeto de la clase Aritmetica
Aritmetica a = new Aritmetica();

//Llamamos el metodo sumar y recibimos el valor devuelto
int resultado = a.sumar(5, 3);
```

Métodos de una clase:

Ejemplo de definicion y llamada de un metodo

Definicion del Metodo

```
public class calcularSueldo {  
    float sueldo;  
    void calculoSuelto (int horas, float pago)  
    {  
        if (horas<=0 || pago<=0) {  
            System.out.println(  
                "No se puede calcular el sueldo.....");  
        }  
        else {  
            sueldo=horas*pago;  
            System.out.println("El sueldo es: "+sueldo);  
        }  
    }  
}
```

Llamada del Metodo

```
import java.util.Scanner;  
public class Sueldo_Prueba {  
    public static void main(String[] args) {  
        // TODO code application logic here  
        int horasTrabajadas;  
        float pagoPorHora;  
        Scanner entrada = new Scanner(System.in);  
        calcularSueldo obj = new calcularSueldo();  
        System.out.print(  
            "Digite el total de horas trabajadas: ");  
        horasTrabajadas=entrada.nextInt();  
        System.out.print(  
            "Digite el pago por cada hora: ");  
        pagoPorHora=entrada.nextFloat();  
        obj.calculoSueldo(horasTrabajadas, pagoPorHora);  
    }  
}
```



UNIVERSIDAD
DON BOSCO

Ventajas del uso de Metodos

■ Encapsulamiento

Ocultan la implementación interna de los objetos y exponen solo la interfaz necesaria, protegiendo el estado interno.

■ Validación

Permiten incluir lógica para verificar datos antes de modificar el estado del objeto, manteniendo la integridad de los datos.

■ Flexibilidad

Facilitan la refactorización; se puede cambiar la implementación interna de un método sin afectar el código cliente que lo usa.

■ Mantenibilidad

Centralizan la lógica de negocio, haciendo el código más fácil de depurar, probar y actualizar.

Anatomía de un Método en Java

```
public class CalculadoraFinanciera { // Clase contenedora
    // Ejemplo de un atributo
    private double saldoActual = 1000.0;
    /**
     * Calcula el interés simple sobre el saldo actual.
     * @param meses El número de meses para calcular el interés.
     * @return El monto del interés calculado.
     */
    public double calcularInteres(int meses) {
        final double TASA_ANUAL = 0.05; // Constante local (final)
        double tasaMensual = TASA_ANUAL / 12;
        double interes = this.saldoActual * tasaMensual * meses;
        // Uso de 'this' para el atributo de instancia
        return interes; // Valor devuelto por el método
    }
    // Método estático principal
    public static void main(String[] args) {
        CalculadoraFinanciera calc = new CalculadoraFinanciera();
        // Creación de un objeto
        double interesGanado = calc.calcularInteres(6);
        System.out.println("Interés ganado en 6 meses: " +
                           interesGanado);
    }
}
```

Constructores(1)



Un constructor es un método especial que se ejecuta automáticamente cuando se crea una nueva instancia de una clase.

Su propósito principal es inicializar el objeto en un estado válido y consistente, asegurando que todos sus atributos tengan valores iniciales apropiados.

Características Fundamentales

- **Mismo nombre que la clase:** El constructor debe tener exactamente el mismo nombre que la clase (PascalCase).
- **Sin tipo de retorno:** No declara ningún tipo de retorno, ni siquiera `void`.
- **Llamado automáticamente:** Se invoca al usar la palabra clave `new` para crear un objeto.
- **Puede ser sobrecargado:** Una clase puede tener múltiples constructores con diferentes firmas (número o tipo de parámetros).
- **Constructor por defecto:** Si no defines ningún constructor explícitamente, el compilador de Java proporciona uno público y vacío automáticamente.

¿Por qué son importantes en Java?

Los constructores en Java aseguran que los objetos nunca existan en un estado inválido o

inconsistente desde el momento de su creación. Permiten:

- Inicializar campos (atributos) con valores apropiados.
- Validar datos de entrada antes de crear el objeto.
- Realizar configuraciones iniciales necesarias (ej. abrir conexiones, cargar recursos).
- Establecer relaciones iniciales con otros objetos.
- Garantizar la integridad de datos desde el inicio de la vida útil del objeto.

Constructores(2)



Constructor por Defecto (Implícito o Explícito)

Un constructor sin parámetros que inicializa los atributos con valores predeterminados o lógicos para la clase.

```
public class CuentaBancaria {
    private double saldo;
    private boolean activa;
    private String numeroCuenta;

    // Constructor explícito por defecto
    public CuentaBancaria() {
        this.saldo = 0.0;
        this.activa = true;
        this.numeroCuenta =
            generarNumeroCuenta();
        System.out.println(
            "Creo nueva cuenta bancaria");
    }

    private String generarNumeroCuenta()
    {
        return "ES" +
            System.currentTimeMillis();
    }
}
```

Constructor Parametrizado

Recibe valores como argumentos para inicializar atributos específicos del objeto.

```
public class CuentaBancaria {
    private String titular;
    private double saldo;
    private boolean activa;
    // ... otros atributos ...
    public CuentaBancaria(
        String titular, double
        saldoInic)
    {
        if (saldoInic < 0) {
            throw new
                IllegalArgumentException(
                    "Saldo no puede ser negativo");
        }
        this.titular = titular;
        this.saldo = saldoInic;
        this.activa = true;
        System.out.println(
            "Cuenta bancaria creada para "
            + titular + " con saldo inicial " +
            saldoInic + ".");
    }
}
```

Constructor de Copia

Crea un nuevo objeto, inicializando sus atributos con los valores de otro objeto existente de la misma clase.

```
public class CuentaBancaria {
    private String titular;
    private double saldo;
    private String tipoCuenta;
    // ... otros atributos ...
    public CuentaBancaria(
        CuentaBancaria otraCuenta) {
        // Constructor de copia
        if (otraCuenta == null) {
            throw new
                IllegalArgumentException(
                    "Cuenta a copiar no puede ser nula.");
        }
        this.titular =
            otraCuenta.titular;
        this.saldo = otraCuenta.saldo;
        this.tipoCuenta =
            otraCuenta.tipoCuenta;
        this.activa = otraCuenta.activa;
        System.out.println(
            "Cuenta bancaria copiada para
            "+
            this.titular + ".");
    }
}
```

Sobrecarga de Métodos: Flexibilidad y Versatilidad(1)

La sobrecarga de métodos (**method overloading**) permite definir múltiples métodos con el mismo nombre pero con diferentes listas de parámetros dentro de la misma clase.

El compilador de Java (`javac`) determina qué versión del método llamar basándose en los tipos y el número de argumentos proporcionados en la llamada.



Firma del Método

Incluye su nombre y el tipo, número y orden de sus parámetros.

Es crucial recordar que el tipo de retorno NO forma parte de la firma para propósitos de sobrecarga.



Diferentes Combinaciones de Parámetros

Puedes sobrecargar métodos variando el número de parámetros, sus tipos (por ejemplo, `int` vs. `double`), o la secuencia de los tipos de sus parámetros, para crear distintas versiones del mismo método.



Conveniencia para el Desarrollador (API)

La sobrecarga proporciona una API más intuitiva, permitiendo múltiples formas de realizar una operación conceptualmente similar, adaptándose a diferentes contextos y necesidades de los usuarios de la clase.

```
public class Calculadora {
    // Método para sumar dos enteros
    public int sumar(int a, int b) {
        return a + b;
    }
    // Método sobrecargado para sumar dos números de punto flotante
    public double sumar(double a, double b) {
        return a + b;
    }
    // Método sobrecargado para sumar un array de enteros
    public int sumar(int[] numeros) {
        int suma = 0;
        for (int num : numeros) suma += num;
        return suma;
    }
    public static void main(String[] args) {
        Calculadora calc = new Calculadora();
        // Llama a sumar(int a, int b)
        System.out.println("Suma de 5 y 3: " + calc.sumar(5, 3));
        // Llama a sumar(double a, double b)
        System.out.println("Suma de 5.5 y 3.2: " + calc.sumar(5.5, 3.2));
        // Llama a sumar(int[] numeros)
        int[] misNumeros = {1, 2, 3, 4};
        System.out.println("Suma de array {1,2,3,4}: " + calc.sumar(misNumeros));
    }
}
```

Sobrecarga de Métodos: Flexibilidad y Versatilidad(2)

Reglas y Consideraciones Clave en Java

1 Diferencia en la Lista de Parámetros

Los métodos sobrecargados DEBEN diferir en el número, tipo o secuencia de los tipos de sus parámetros. Es la única forma en que el compilador puede distinguirlos.

2 El Tipo de Retorno No Cuenta

No se puede sobrecargar un método solo cambiando su tipo de retorno.

El compilador generará un error.

3 Promoción Automática de Tipos

Java puede promover tipos de datos automáticamente (por ejemplo, un int a un double). Esto puede influir en la resolución de la sobrecarga.

4 Evitar Ambigüedad

Es importante evitar crear sobrecargas que el compilador de Java no pueda distinguir de manera inequívoca, ya que esto resultará en errores de compilación.

Ventaja de sobrecargar metodos: Mejora drásticamente la legibilidad y usabilidad del código al permitir usar el mismo nombre para operaciones conceptualmente similares, sin importar los tipos de datos exactos involucrados, haciendo las APIs más fáciles de aprender y usar.

Paradigmas de POO

Es un modelo de diseño de software que organiza el código en objetos, que son **entidades** que combinan **datos** (atributos) y **comportamiento** (métodos), simulando el mundo real para facilitar la reutilización, mantenibilidad y el trabajo en equipo.

Principios de la POO

- **Abstracción:** Enfocarse en lo esencial de un objeto, ocultando detalles irrelevantes para el contexto actual y modelando entidades del mundo real.
- **Encapsulamiento:** Agrupar datos y métodos dentro del objeto, protegiendo la información interna y controlando el acceso a ella.
- **Herencia:** Permitir que una clase adquiera atributos y métodos de otra clase, promoviendo la reutilización de código.
- **Polimorfismo:** Capacidad de objetos de diferentes clases para responder al mismo mensaje (método) de maneras distintas, adaptándose al contexto.

PОО: Principio del Encapsulamiento

- Consiste en ocultar los detalles de la implementación de un objeto, a la vez que se provee una interfaz pública por medio de sus métodos permitidos.
- Se puede ver como una "caja negra" donde el exterior del Objeto interactúa solo con lo esencial (los métodos públicos), sin preocuparse por cómo funcionan internamente los datos, similar a un coche donde usas el volante pero no manipulas directamente el motor.

Metodos de Propiedad (Accesores)

- Encapsulamiento tiene la siguiente *regla de oro* para implementarlo: *atributos deben ser privados y los métodos públicos*.
- Para acceder públicamente a los campos privados, se crean métodos denominados **Metodos de Propiedad** (o simplemente **Propiedades** o **accesores**).

POO: Principio del Encapsulamiento



Getters (Accesor de lectura)

Método que permite leer el valor de un atributo privado sin exponerlos directamente.

Proporcionan acceso controlado de solo lectura a los datos internos del objeto.

En Java, suelen seguir la convención `getNombreDelAtributo()`.

```
public class MiClase {  
    private String nombre;  
    private double saldo;  
    public String getNombre() {  
        return this.nombre;  
    }  
    // 'this' se refiere a instancia actual del objeto  
    public double getSaldo() {  
        return this.saldo;  
    }  
}
```



Setters (Accesor de escritura)

Métodos que permiten modificar atributos privados de manera controlada.

Pueden incluir lógica de validación para asegurar que solo se asignen valores válidos.

En Java, suelen seguir la convención `setNombreDelAtributo(Tipo valor)`.

```
public class MiClase {  
    private double saldo;  
    private int edad;  
    public void setSaldo(double nuevoSaldo) {  
        if (nuevoSaldo >= 0) {  
            this.saldo = nuevoSaldo;  
        } else  
            System.out.println(  
                "Error: El saldo no puede ser negativo.");  
    }  
    public void setEdad(int edad) {  
        if (edad > 0 && edad < 150) // Validación de rango  
            this.edad = edad;  
        else  
            System.out.println(  
                "Error: Edad fuera de rango válido.");  
    }  
}
```

Convención de Nombres en Java:

Los nombres de métodos deben usar `camelCase` y comenzar con un verbo que describa la acción (ej: `calcular`, `obtener`, `establecer`, `validar`, `procesar`).

Los nombres de clases usan `PascalCase`.

Representación de Clases en UML(1)

Estructura del Diagrama de Clase



Una clase en UML se representa con un rectángulo dividido en tres compartimentos horizontales, reflejando la estructura de una clase Java:

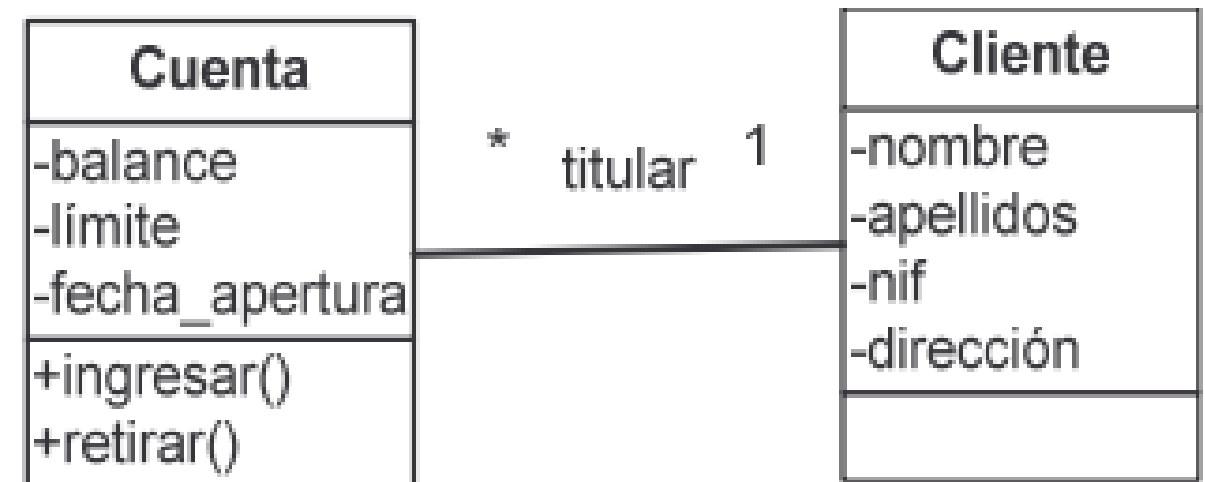
1. **Nombre de la Clase:** En la parte superior, generalmente en PascalCase (como en Java), en negrita y centrado.
2. **Atributos (Campos/Variables de Instancia):** En el compartimento medio, incluyendo el nombre (en camelCase), el tipo de dato (String, int, double, etc.) y el modificador de acceso.
3. **Métodos:** En el ultimo compartimento. Incluyen el nombre del método (en camelCase), sus parámetros (tipo y nombre) y el tipo de retorno.



Símbolos de Visibilidad en Java

Los modificadores de acceso en Java se representan con los siguientes símbolos especiales antes del nombre en UML:

- + (plus)** = public: acceso público sin restricciones desde cualquier clase.
- (minus)** = private: acceso solo dentro de la clase que lo declara.
- # (hash)** = protected: acceso en la clase, subclasses y clases del mismo paquete.
- ~ (tilde)** = default/package-private: acceso solo dentro del mismo paquete (no explícito en Java con palabra clave, es la ausencia de modificador).



Representación de Clases en UML(2)



Ejemplo diagrama UML de una Clase

Notación Detallada para Java

Formato de Atributos:

```
visibilidad nombreAtributo: TipoDato = valorInicial
```

Ejemplo:

```
- edad: int = 0    su declaracion en Java sería private int edad = 0;
```

Formato de Métodos:

```
visibilidad nombreMetodo(param1: Tipo1, param2: Tipo2): TipoRetorno
```

Ejemplo:

```
+ calcular(x: int, y: int): double
```

La codificación en Java sería: `public double calcular(int x, int y)`

Indicadores Especiales en UML:

* Subrayado = miembro estático (`static` en Java, compartido por todas las instancias de la clase).

* *Cursiva* = clase o método abstracto (`abstract` en Java, no implementado o con una interfaz).

* **{readonly}** = campo de solo lectura (`final` en Java, su valor no puede ser modificado después de la inicialización).

CuentaBancaria
-numeroCuenta: String -titular: String -saldo: double -activa: boolean +COMISION: double = 0.15 {readOnly}
+CuentaBancaria(titular: String, saldo: double) +depositar(monto: double): void +retirar(monto: double): boolean +getSaldo(): double -validarMonto(monto: double): boolean

Ejemplo Completo en Código Java y su Representación UML



```
package com.example.model; // Paquete Java
public class Estudiante {
    // Atributos (Variables de instancia)
    private String nombre;
    private int matricula;
    protected double promedio;
    public static int totalEstudiantes=0; // Atributo estático
    // Métodos
    public Estudiante(String nombre, int matricula) { // Constructor
        this.nombre = nombre;
        this.matricula = matricula;
        this.promedio = 0.0; // Valor por defecto
        totalEstudiantes++; // Incrementa el contador estático
    }
    public void actualizarPromedio(double nuevoPromedio) {
        if (nuevoPromedio >= 0 && nuevoPromedio <= 10)
            this.promedio = nuevoPromedio;
    }
    public String getNombre() {
        return nombre;
    }
    private boolean esAprobado() { // Método privado
        return promedio >= 6.0;
    }
    // Ejemplo de método con ArrayList
    public void registrarNotas(java.util.ArrayList<Double> notas) {
        // Lógica para procesar las notas
    }
}
```

Estudiante
-nombre: String -matricula: int #promedio: double <u>+totalEstudiantes: int</u>
+Estudiante(nombre: String, matricula: int) +actualizarPromedio(nuevoPromedio: double): void +getNombre(): String -esAprobado(): boolean +registrarNotas(notas: ArrayList<Double>): void

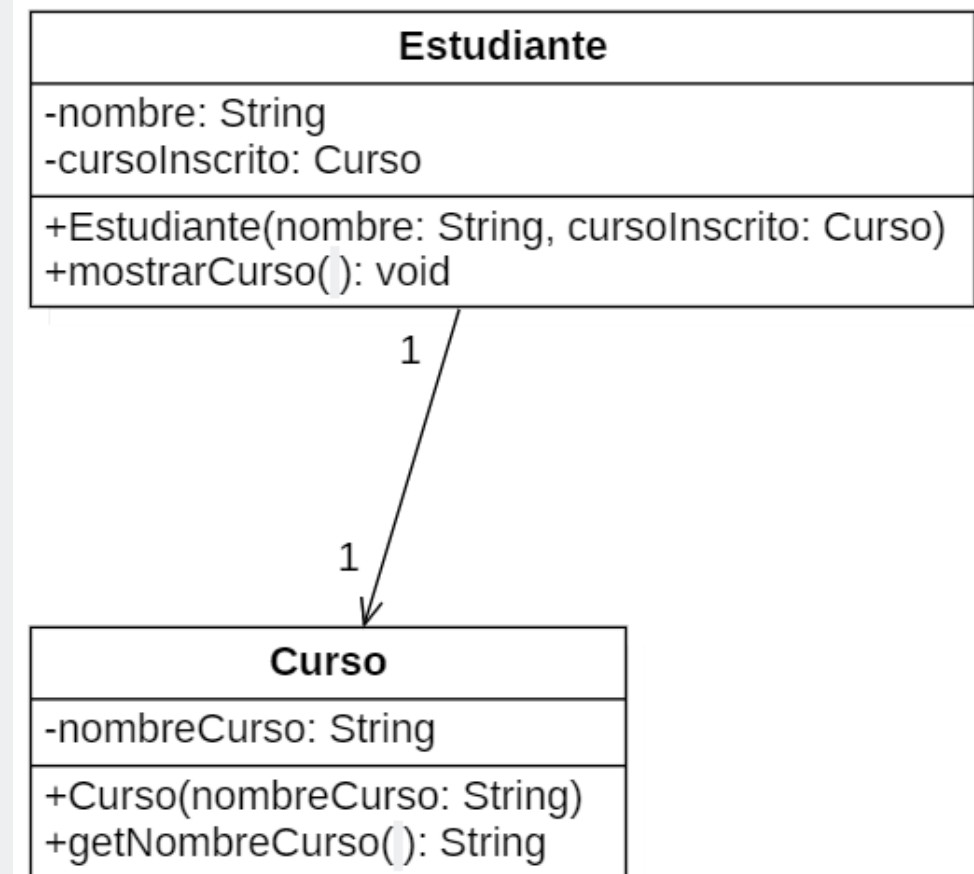
Relaciones entre Clases en UML

➡ Asociación (Association)

La asociación es una relación básica donde una clase utiliza o interactúa con otra. En Java, esto se representa comúnmente cuando un objeto de una clase tiene una referencia a un objeto de otra clase. La relación puede ser unidireccional o bidireccional.

// Ejemplo de Asociación Unidireccional en Java

```
class Curso {  
    private String nombreCurso;  
    // ... otros atributos y métodos  
    public Curso(String nombreCurso) { //constructor  
        this.nombreCurso = nombreCurso;  
    }  
    public String getNombreCurso() {  
        return nombreCurso;  
    }  
}  
  
class Estudiante {  
    private String nombre;  
    private Curso cursoInscrito;  
    // Asociación: Estudiante 'tiene' un Curso  
    public Estudiante(String nombre, Curso cursoInscrito) {  
        this.nombre = nombre;  
        this.cursoInscrito = cursoInscrito;  
    }  
    public void mostrarCurso() {  
        if (cursoInscrito != null)  
            System.out.println(this.nombre + " está inscrito en: " + cursoInscrito.getNombreCurso());  
        else  
            System.out.println(this.nombre + " no tiene un curso inscrito.");  
    }  
}
```

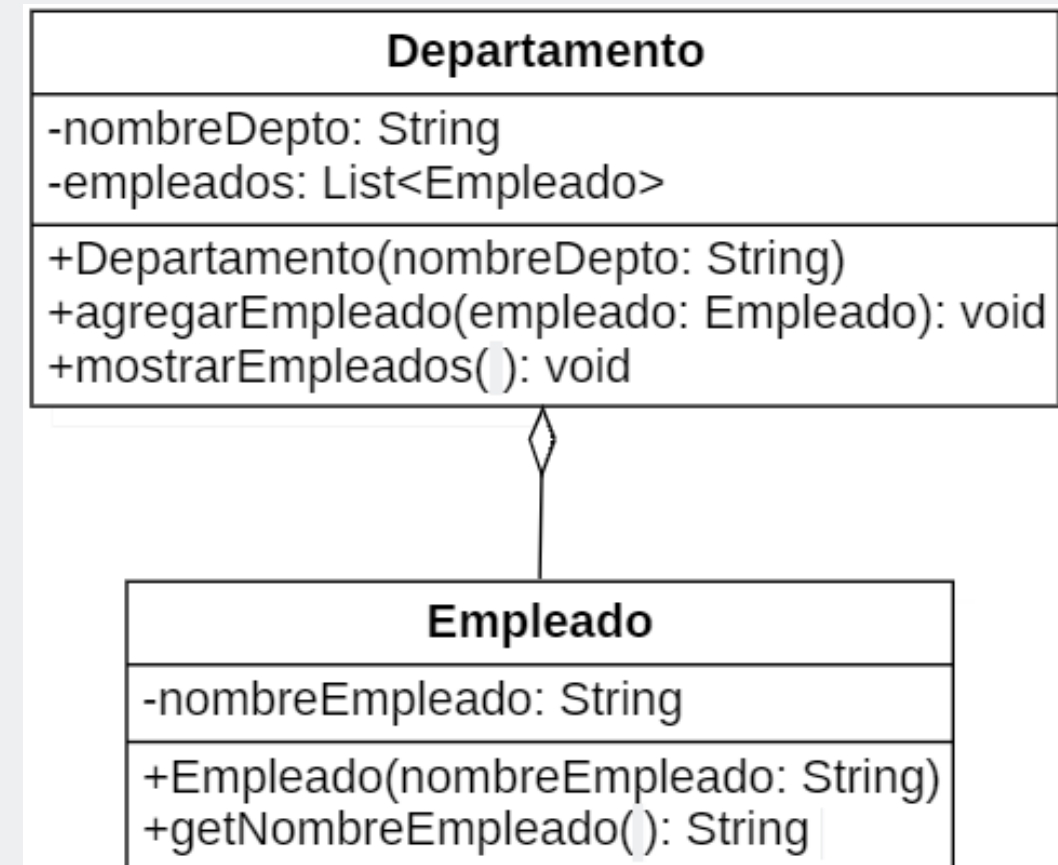




Agregación (Aggregation)-

La agregación es una relación "tiene-un" (has-a) débil, donde el objeto "parte" puede existir independientemente del objeto "todo". Si el objeto "todo" es destruido, el objeto "parte" puede seguir existiendo. Se representa con un diamante blanco hueco en el lado del "todo".

```
// Ejemplo de Agregación en Java
import java.util.List;
import java.util.ArrayList;
class Empleado {
    private String nombreEmpleado;
    // ... otros atributos y métodos
    public Empleado(String nombreEmpleado) {
        this.nombreEmpleado = nombreEmpleado;
    }
    public String getNombreEmpleado() {
        return nombreEmpleado;
    }
}
class Departamento {
    private String nombreDepto;
    private List<Empleado> empleados;
    // Agregación: Departamento 'tiene' Empleados
    public Departamento(String nombreDepto) {
        this.nombreDepto = nombreDepto;
        // Crea la lista, pero los empleados existen fuera
        this.empleados = new ArrayList<>();
    }
    public void agregarEmpleado(Empleado empleado) {
        this.empleados.add(empleado);
    }
    public void mostrarEmpleados() {
        System.out.println("Departamento: " + nombreDepto);
        if (empleados.isEmpty())
            System.out.println(" No hay empleados asignados.");
        else
            for (Empleado emp : empleados)
                System.out.println(" - " + emp.getNombreEmpleado());
    }
}
```

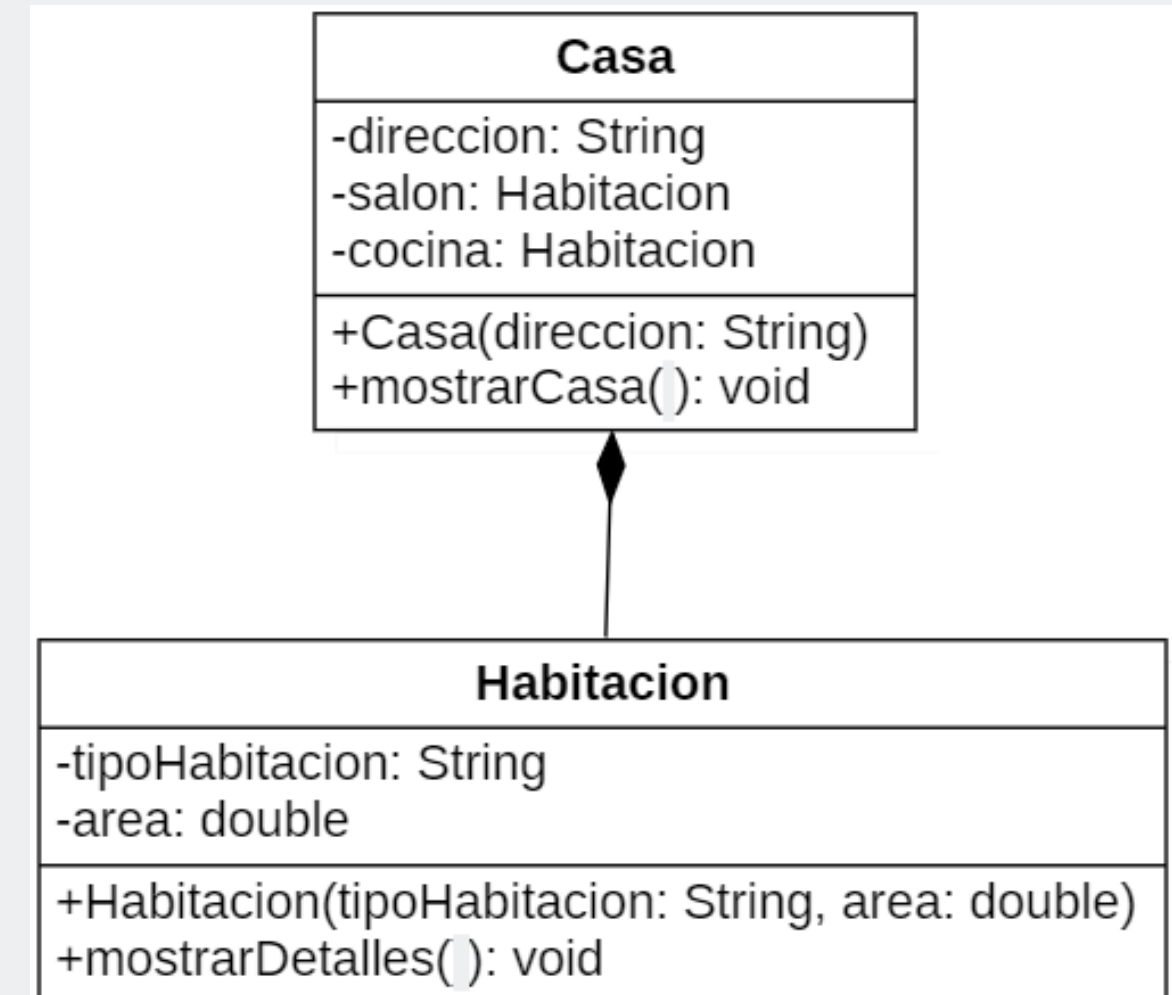


Composición (Composition)

La composición es una relación "tiene-un" (has-a) fuerte, donde el objeto "parte" no puede existir sin el objeto "todo". Si el objeto "todo" es destruido, el objeto "parte" también es destruido. Se representa con un diamante negro relleno en el lado del "todo".

```
// Ejemplo de Composición en Java
class Habitacion {
    private String tipoHabitacion;
    private double area;
    public Habitacion(String tipoHabitacion, double area) {
        this.tipoHabitacion = tipoHabitacion;
        this.area = area;
    }
    public void mostrarDetalles() {
        System.out.println(" Tipo: " + tipoHabitacion + ", Área: " + area + " m²");
    }
}

class Casa {
    private String direccion;
    private Habitacion salon;
    // Composición: La casa 'crea' sus propias habitaciones
    private Habitacion cocina;
    public Casa(String direccion) {
        this.direccion = direccion;
        this.salon = new Habitacion("Salón", 30.5);
        this.cocina = new Habitacion("Cocina", 15.0);
    }
    public void mostrarCasa() {
        System.out.println("Casa en: " + direccion);
        System.out.println("Habitaciones:");
        salon.mostrarDetalles();
        cocina.mostrarDetalles();
    }
}
```



Dependencia (Dependency)

La dependencia es una relación de uso más débil y temporal, donde una clase utiliza a otra, pero no la mantiene como un atributo a largo plazo.

Normalmente ocurre cuando una clase "depende" de otra para completar una tarea, a menudo pasando un objeto de la otra clase como parámetro a un método o creándolo localmente.

```
// Ejemplo de Dependencia en Java
import java.util.Scanner; // Clase Scanner es un ejemplo común de dependencia
class LectorEntrada {
    public String leerLinea() {
        // La clase LectorEntrada depende de Scanner para leer la entrada
        Scanner scanner = new Scanner(System.in);
        System.out.print("Introduce un texto: ");
        String entrada = scanner.nextLine();
        scanner.close(); // Es importante cerrar recursos
        return entrada;
    }
}

class ProcesadorTexto {
    public void procesar(LectorEntrada lector) { // Dependencia a través de un parámetro
        String texto = lector.leerLinea();
        System.out.println("Texto procesado: " + texto.toUpperCase());
    }
}
```